ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA KỸ THUẬT MÁY TÍNH



BÁO CÁO THỰC HÀNH LAB 03

MÔN HỌC: THIẾT KẾ LUẬN LÝ SỐ

ĐỀ TÀI

THIẾT KẾ MẠCH TỔ HỢP PHỤC VỤ TÍNH TOÁN

(ALU 16 BIT VÀ BỘ NHÂN HAI SỐ 4 BIT)

Giảng viên hướng dẫn: Nguyễn Thành Nhân
Sinh viên thực hiện: Huỳnh Phạm Ngọc Thắng
Mã số sinh viên: 25521672
Lớp: CE118.R11.VMTN.1

TP. Hồ Chí Minh, tháng 11 năm 2026

1. GIỚI THIỆU ĐỀ TÀI

1.1. Yêu cầu bài tập

Lab03 gồm hai mạch tổ hợp vẽ bằng Block Diagram trên Quartus, chỉ mô phỏng **Functional** và nạp kiểm tra trên **kit DE2**:

- **Phần thực hành – ALU 16 bit:** hai toán hạng A, B 16 bit, tín hiệu **Opcode[2..0]** chọn 8 phép toán theo thứ tự 0 → 7: cộng, cộng 1, trừ, trừ 1, AND, OR, NAND, XOR. ALU gồm khối **AU** (số học), khối **LU** (logic) và bộ **MUX** chọn kết quả theo Opcode[2].
- **Phần bài tập – Bộ nhân 4 bit:** thiết kế theo hướng **bộ dịch và bộ cộng**. Đề yêu cầu nhân với số cuối MSSV; bài này **mở rộng thành nhân hai số 4 bit bất kỳ** A × B, kết quả 8 bit (lớn nhất 15 × 15 = 225).

1.2. Thông số thiết kế

Bảng 1.1: Thông số thiết kế

Thông số	ALU 16 bit	Bộ nhân 4 bit
Tín hiệu vào	A[15..0], B[15..0], Opcode[2..0]	A[3..0], B[3..0]
Tín hiệu ra	Y[15..0]	Tich[7..0]
Loại mạch	Tổ hợp	Tổ hợp (dịch + cộng)
Mô phỏng	Functional	Functional
Công cụ	Quartus II, kit DE2	Quartus II, kit DE2

Ghi chú: Kit DE2 chỉ có 18 công tắc gạt, không đủ cho hai số 16 bit. Vì vậy khi nạp ALU, **10 bit cao** của A và B được giữ cố định (nối VCC/GND ngay trong mạch), chỉ **6 bit thấp** A[5..0], B[5..0] và Opcode được thay đổi bằng công tắc.

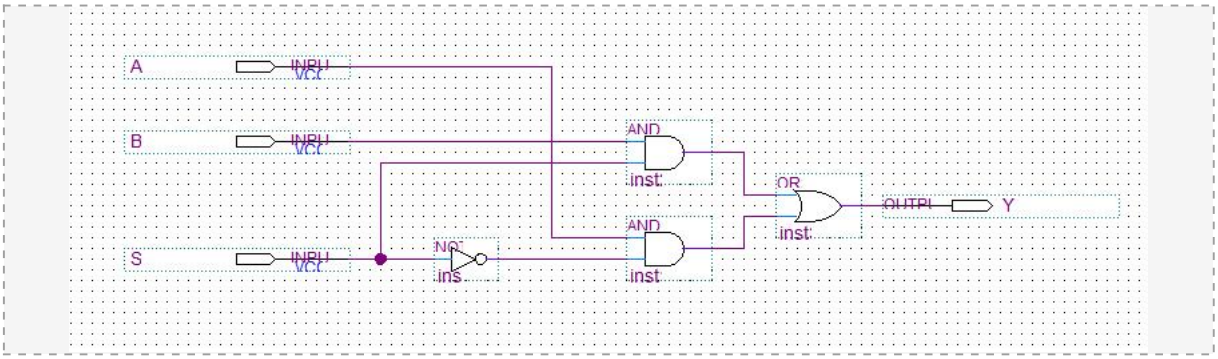
2. CÁC KHỐI CƠ BẢN

Cả hai mạch được xây từ các khối nhỏ dưới đây, vẽ hoàn toàn bằng cổng logic.

2.1. MUX 2-1 (mux2_1)

$$Y = S' \cdot A + S \cdot B$$

S = 0 → Y = A; S = 1 → Y = B. Mạch gồm 1 cổng NOT, 2 cổng AND và 1 cổng OR. Ghép 16 khối dùng chung S ta được MUX 2-1 16 bit (file mux2_1_8bit trong project ALU, ngõ vào/ra 16 bit).



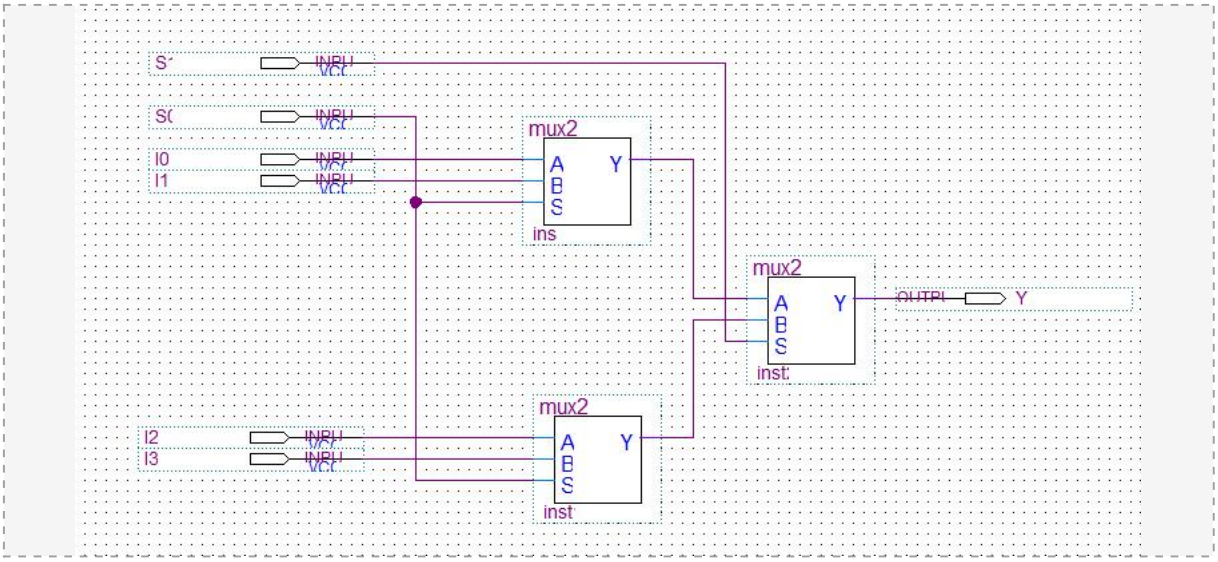
Hình 2.1: Mạch MUX 2-1

2.2. MUX 4-1 (mux4_1)

Ghép từ 3 bộ MUX 2-1: hai bộ đầu dùng S0 chọn trong cặp (I0, I1) và (I2, I3), bộ cuối dùng S1 chọn giữa hai kết quả.

Bảng 2.1: Bảng chức năng MUX 4-1

S1	S0	Y
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3



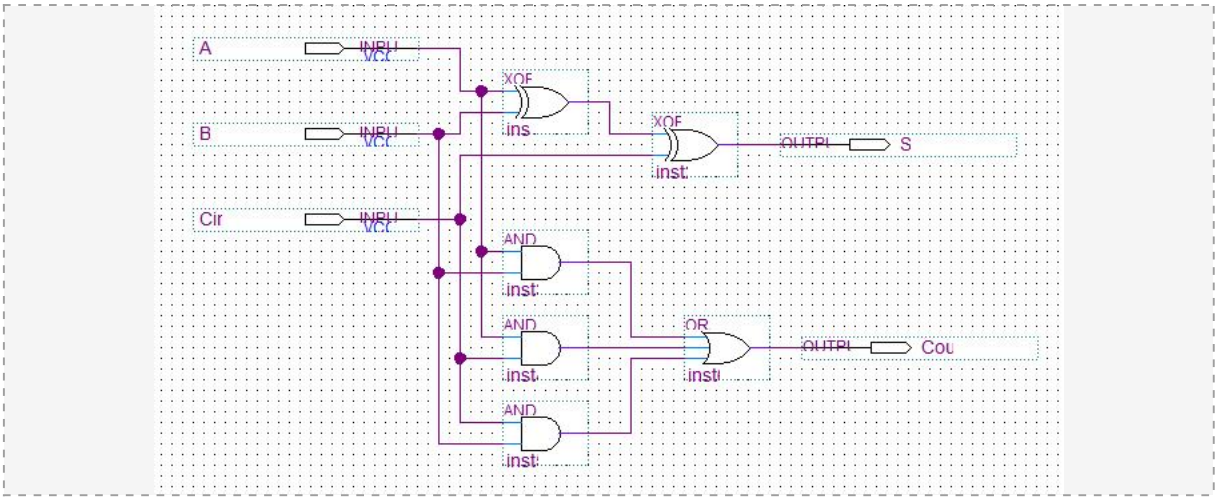
Hình 2.2: Mạch MUX 4-1

2.3. Bộ cộng toàn phần (full_adder)

$$S = A \oplus B \oplus C_{in}$$

$$C_{out} = A \cdot B + A \cdot C_{in} + B \cdot C_{in}$$

Mạch gồm 2 cổng XOR, 3 cổng AND và 1 cổng OR3.



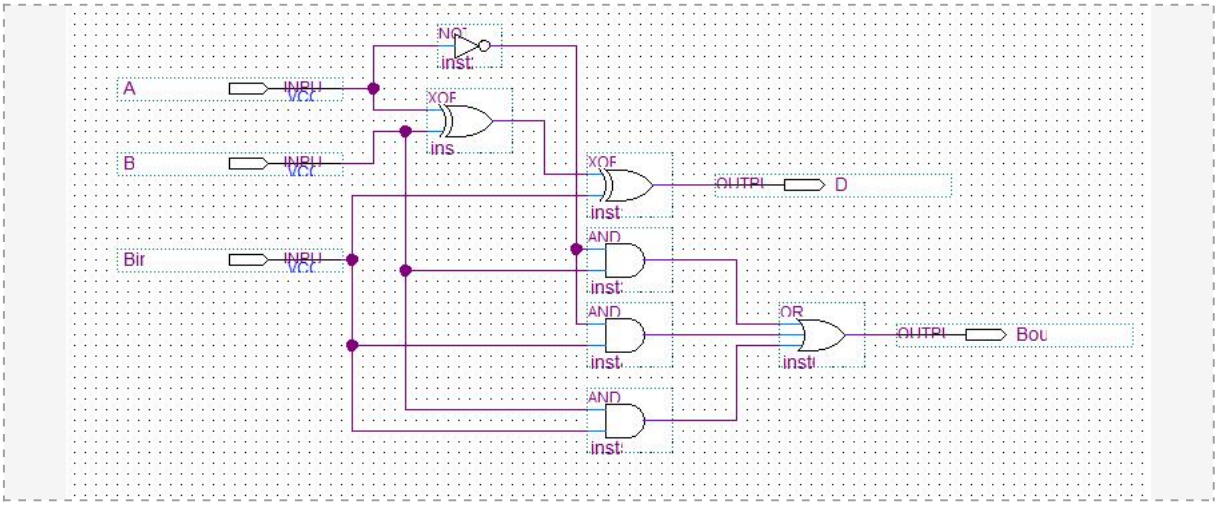
Hình 2.3: Mạch Full adder

2.4. Bộ trừ toàn phần (full_sub)

$$D = A \oplus B \oplus B_{in}$$

$$B_{out} = A' \cdot B + A' \cdot B_{in} + B \cdot B_{in}$$

Cấu trúc giống bộ cộng, nhưng A được đảo qua cổng NOT trước khi vào các cổng AND tính số mượn.



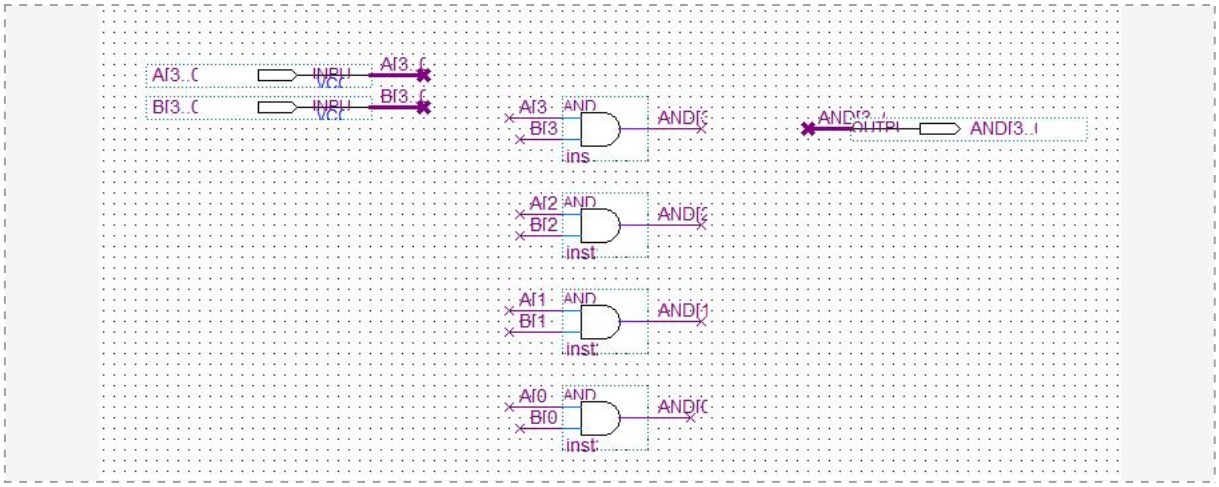
Hình 2.4: Mạch Full subtractor

2.5. Cổng logic 4 bit và 16 bit

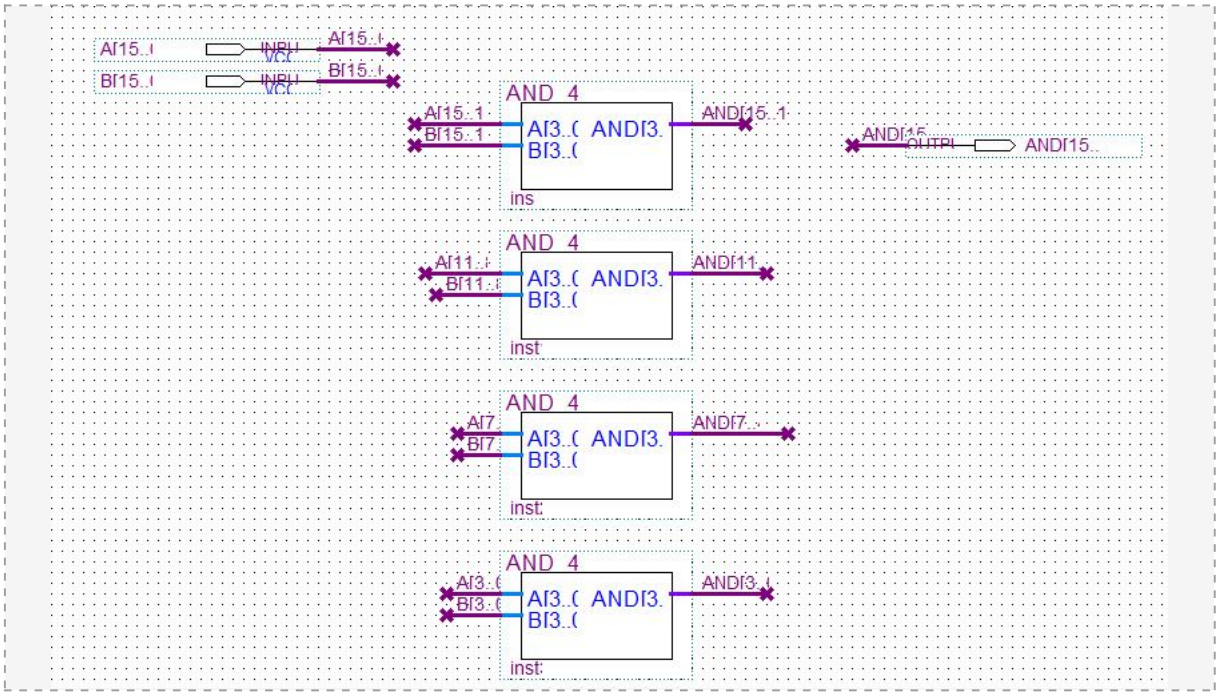
Mỗi khối 4 bit gồm 4 cổng 2 ngõ vào xử lý từng cặp bit A[i], B[i]. Khối 16 bit ghép 4 khối 4 bit cho các nhóm bit [15..12], [11..8], [7..4], [3..0].

Bảng 2.2: Các khối logic của LU

Khối 4 bit	Khối 16 bit	Cổng dùng	Ngõ ra
AND_4bit	AND_16bit	4 × AND2	AND[15..0]
OR_4bit	OR_16bit	4 × OR2	OR[15..0]
NAND_4bit	NAND_16bit	4 × NAND2	NAND[15..0]
XOR_4bit	XOR_16bit	4 × XOR	XOR[15..0]



Hình 2.5: Khối AND 4 bit



Hình 2.6: Khối AND 16 bit (ghép 4 khối 4 bit)

3. ALU 16 BIT

ALU chọn 1 trong 8 phép toán theo Opcode[2..0]: **Opcode[2]** chọn nhóm (AU hay LU), **Opcode[1..0]** chọn phép trong nhóm.

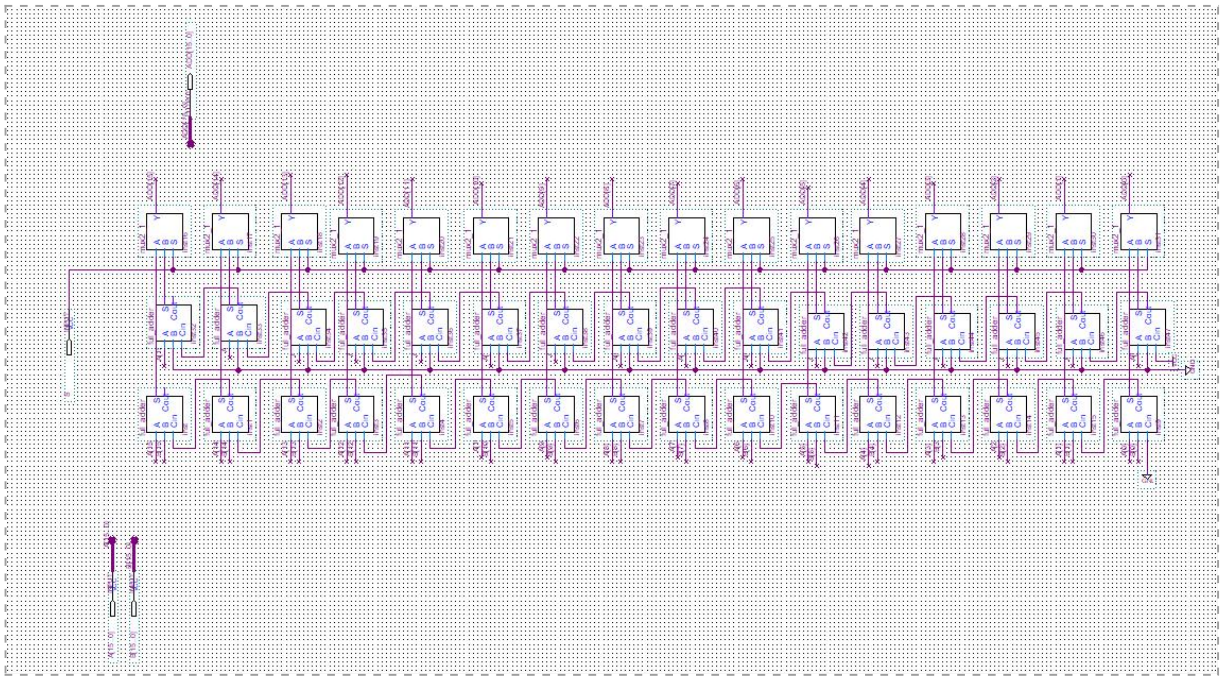
Bảng 3.1: Bảng chức năng ALU

Opcode (S2 S1 S0)	Khối	Phép toán	Kết quả Y
000 (0)	AU	Cộng	A + B
001 (1)	AU	Cộng 1	A + 1
010 (2)	AU	Trừ	A − B
011 (3)	AU	Trừ 1	A − 1
100 (4)	LU	AND	A AND B
101 (5)	LU	OR	A OR B
110 (6)	LU	NAND	A NAND B
111 (7)	LU	XOR	A XOR B

3.1. Bộ cộng 16 bit (full_adder_16bit)

Gồm hai chuỗi 16 full_adder nối tiếp (ripple carry), C_{in} của bit 0 nối GND:

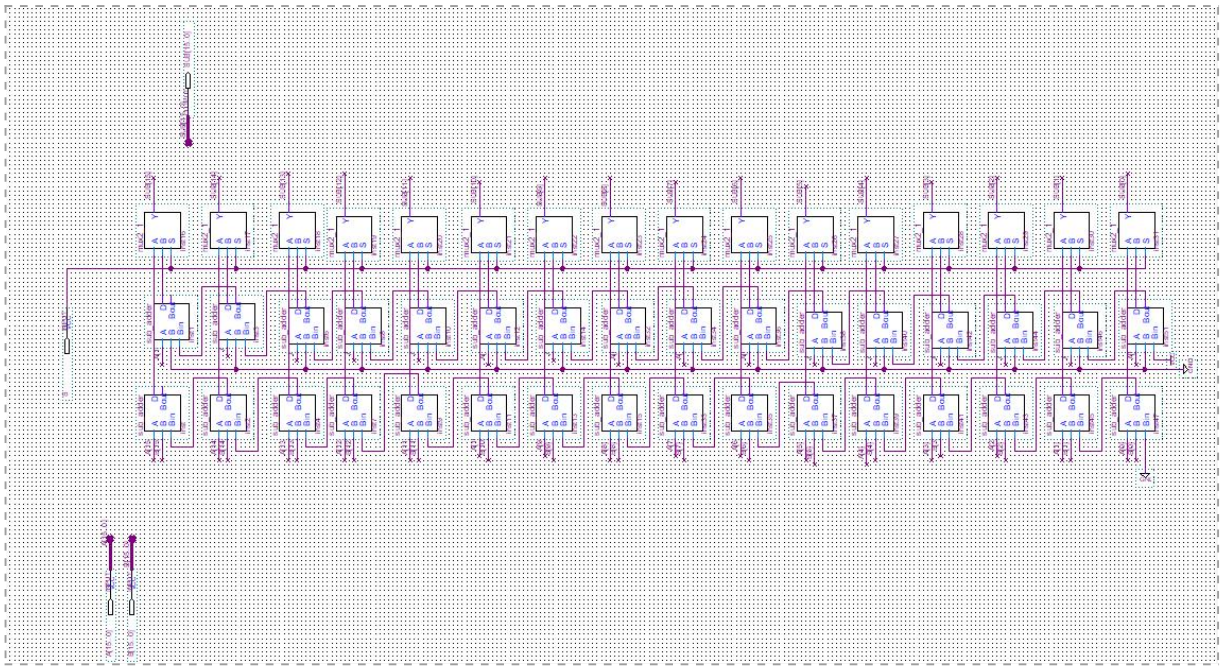
- Chuỗi 1 tính **A + B**.
- Chuỗi 2 tính **A + 1**: ngõ B được thay bằng hằng số 0000 0000 0000 0001 (bit 0 nối VCC, các bit còn lại nối GND).
- 16 bộ mux2_1 dùng ngõ **S** (nối S0) chọn kết quả: S = 0 → A + B, S = 1 → A + 1. Ngõ ra ADD[15..0].



Hình 3.1: Bộ cộng 16 bit

3.2. Bộ trừ 16 bit (full_sub_16bit)

Cấu trúc giống bộ cộng nhưng dùng 32 khối sub_adder: chuỗi 1 tính **A − B**, chuỗi 2 tính **A − 1**, B_{in} của bit 0 nối GND. 16 bộ mux2_1 dùng S (nối S0) chọn: S = 0 → A − B, S = 1 → A − 1. Ngõ ra SUB[15..0]. Khi A < B, kết quả ở dạng bù 2 trên 16 bit.



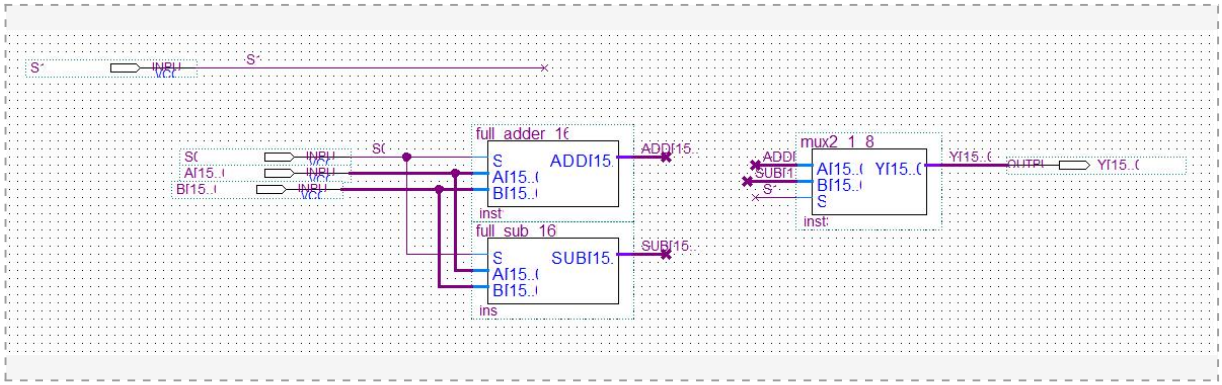
Hình 3.2: Bộ trừ 16 bit

3.3. Khối số học AU

S0 đưa vào ngõ S của cả bộ cộng và bộ trừ; MUX 2-1 16 bit dùng **S1** chọn ADD (S1 = 0) hoặc SUB (S1 = 1).

Bảng 3.2: Bảng chức năng khối AU

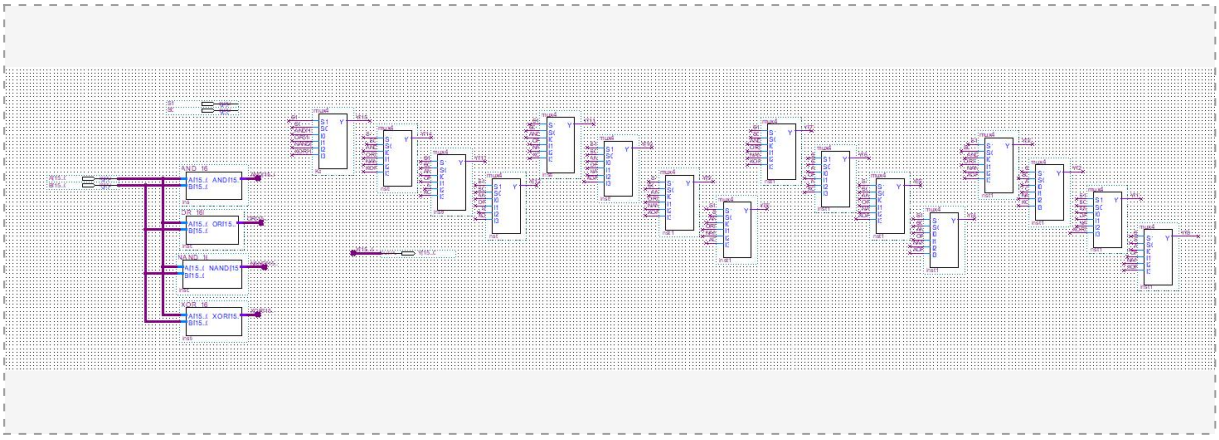
S1	S0	Bộ cộng (ADD)	Bộ trừ (SUB)	Y
0	0	A + B	A − B	A + B
0	1	A + 1	A − 1	A + 1
1	0	A + B	A − B	A − B
1	1	A + 1	A − 1	A − 1



Hình 3.3: Khối số học AU

3.4. Khối logic LU

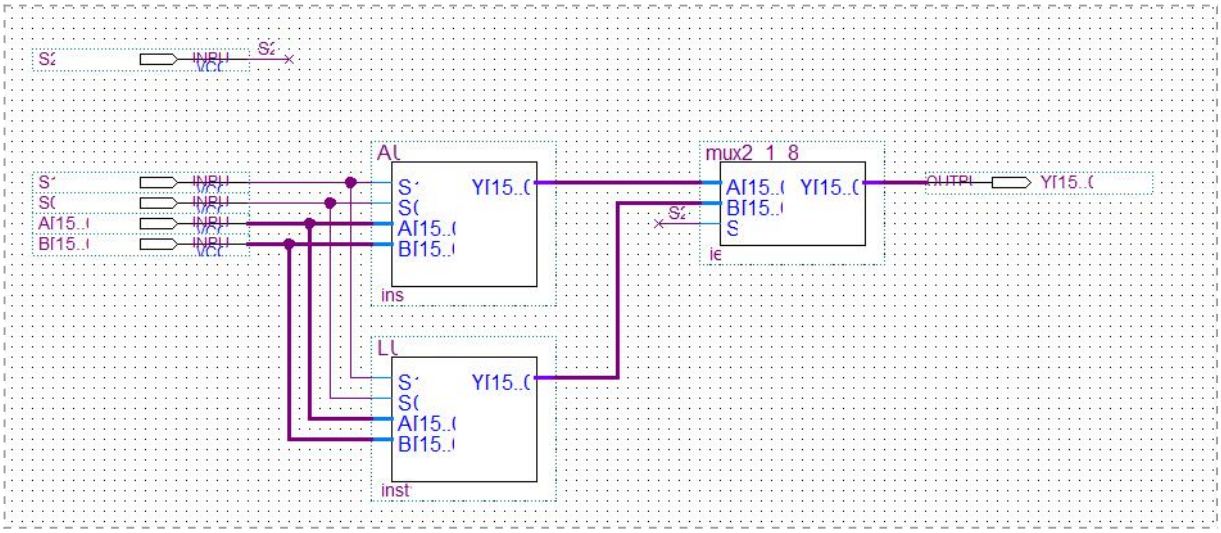
Bốn khối AND_16bit, OR_16bit, NAND_16bit, XOR_16bit tính song song. Mỗi bit ra Y[i] đi qua 1 bộ mux4_1 (tổng 16 bộ) với I_0 = AND[i], I_1 = OR[i], I_2 = NAND[i], I_3 = XOR[i], chọn bởi S1 S0.



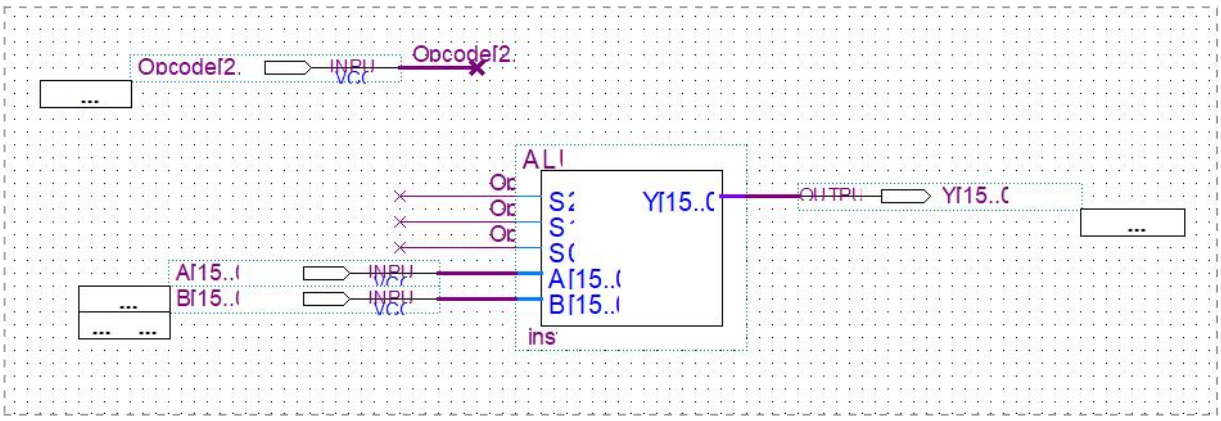
Hình 3.4: Khối logic LU

3.5. Mạch ALU tổng

AU và LU cùng nhận S1, S0, A, B. MUX 2-1 16 bit dùng **S2** chọn kết quả: S2 = 0 → AU, S2 = 1 → LU. Mạch cuối (mach_cuoi) tách bus Opcode[2..0] thành S2, S1, S0 đưa vào ALU.



Hình 3.5: Mạch ALU 16 bit



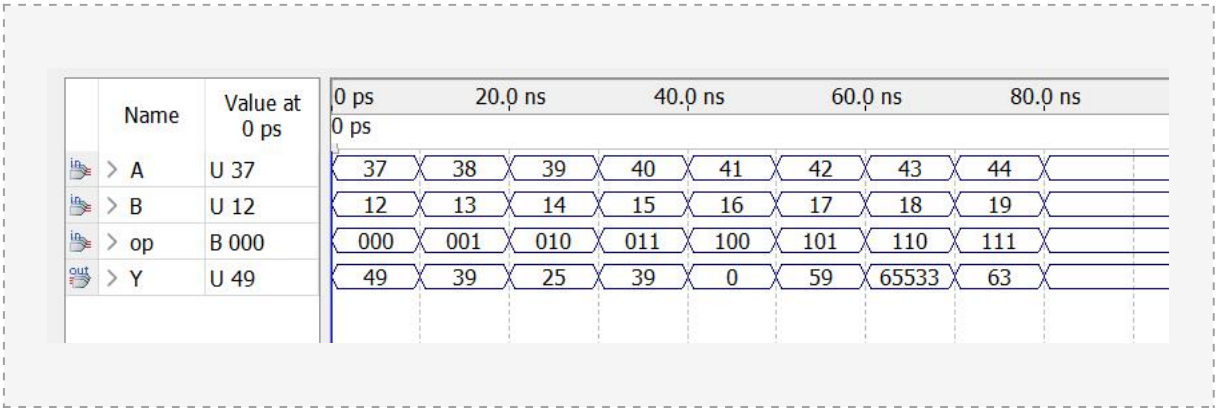
Hình 3.6: Mạch cuối ALU (Opcode[2..0], A[15..0], B[15..0] → Y[15..0])

3.6. Mô phỏng Functional

Mô phỏng mạch cuối với Opcode chạy lần lượt 0 → 7, mỗi giá trị giữ 10 ns (Waveform4.vwf). A, B đổi theo từng khoảng để kiểm tra cả 8 phép toán.

Bảng 3.3: Các trường hợp kiểm tra ALU

Thời gian	Opcode	A	B	Phép toán	Y mong đợi	Kết quả
0 – 10 ns	000	37	12	A + B	49	Đúng
10 – 20 ns	001	38	13	A + 1	39	Đúng
20 – 30 ns	010	39	14	A − B	25	Đúng
30 – 40 ns	011	40	15	A − 1	39	Đúng
40 – 50 ns	100	41 (101001)	16 (010000)	AND	0 (000000)	Đúng
50 – 60 ns	101	42 (101010)	17 (010001)	OR	59 (111011)	Đúng
60 – 70 ns	110	43 (101011)	18 (010010)	NAND	65533 (1111 1111 1111 1101)	Đúng
70 – 80 ns	111	44 (101100)	19 (010011)	XOR	63 (111111)	Đúng



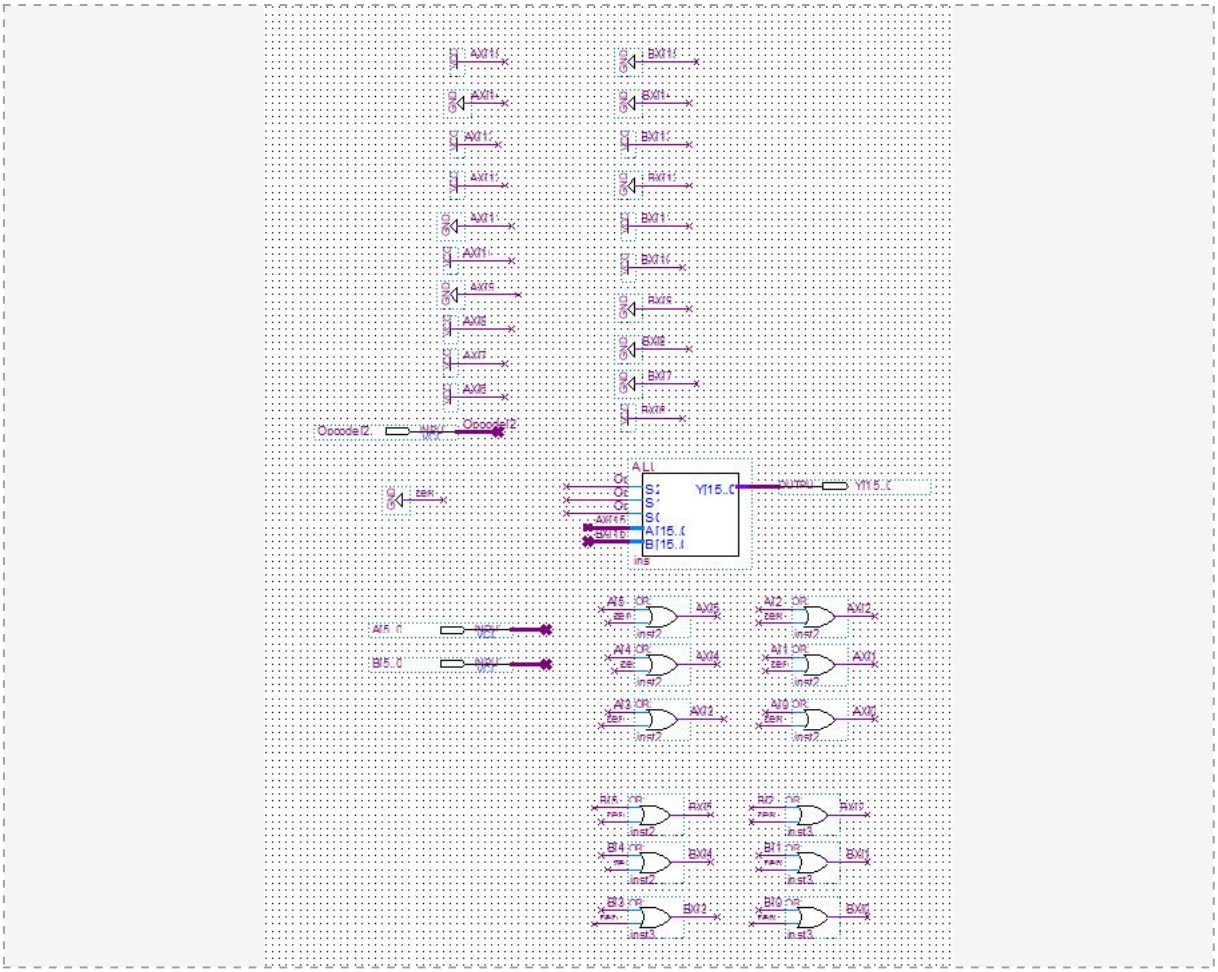
Hình 3.7: Waveform Functional – ALU 16 bit

3.7. Nạp kiểm tra trên kit DE2

Mạch nạp kit (mach_cuoi_duy_nhat) giữ cố định 10 bit cao của A và B bằng VCC/GND, chỉ đưa A[5..0], B[5..0] và Opcode[2..0] ra công tắc; kết quả Y[15..0] hiển thị trên LED đỏ. Cách gán chân và các trường hợp thử được trình bày trong video.

Bảng 3.4: Giá trị cố định khi nạp kit

Tín hiệu	10 bit cao (cố định)	6 bit thấp
A[15..0]	A[15..6] = 1011010111	A[5..0] gạt bằng công tắc
B[15..0]	B[15..6] = 0010110001	B[5..0] gạt bằng công tắc



Hình 3.8: Mạch nạp kit ALU

> **Link video:** Video nạp ALU trên kit DE2:
> https://drive.google.com/file/d/1P83vlCczHYaQyjFbt9l5LfGvQGPH-sEK/view?usp=sharing

4. BỘ NHÂN HAI SỐ 4 BIT

Bộ nhân tính Tich[7..0] = A[3..0] × B[3..0] với A, B bất kỳ (0 – 15), thay vì nhân với một hằng số cố định như đề yêu cầu. Mạch là mạch tổ hợp, đúng hướng **bộ dịch + bộ cộng**.

4.1. Ý tưởng

Xét từng bit B[i] của số nhân: nếu B[i] = 1 thì cộng A đã dịch trái i bit vào tích, nếu B[i] = 0 thì cộng 0.

$$\mathbf{Tich = B_0 \cdot A + B_1 \cdot (A << 1) + B_2 \cdot (A << 2) + B_3 \cdot (A << 3)}$$

Ví dụ với A = 1101 (13), B = 1010 (10), kết quả đúng là 13 × 10 = 130:

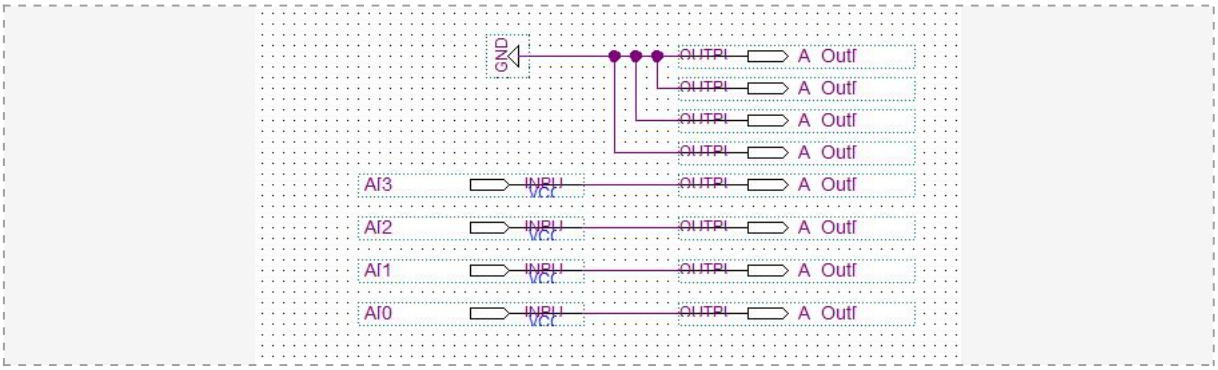
Bảng 4.1: Các tích riêng phần với 1101 × 1010

Bit	B[i]	A dịch trái i bit	Tích riêng phần
i = 0	0	0000 1101 (13)	0000 0000 (0)
i = 1	1	0001 1010 (26)	0001 1010 (26)
i = 2	0	0011 0100 (52)	0000 0000 (0)
i = 3	1	0110 1000 (104)	0110 1000 (104)
Tổng			**1000 0010 (130)**

Tích lớn nhất 15 × 15 = 225 < 256 nên 8 bit là đủ, các C_{out} của bộ cộng không dùng.

4.2. Khối mở rộng InA

Mở rộng A 4 bit lên 8 bit: A_Out[3..0] = A[3..0], A_Out[7..4] nối GND. Đây cũng là tích riêng phần của bit B[0] (A × 1).



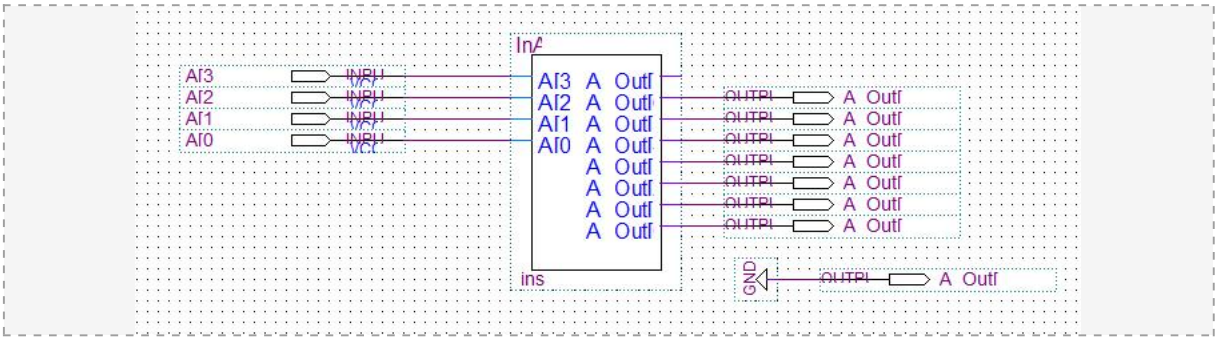
Hình 4.1: Khối mở rộng InA

4.3. Các bộ dịch trái (shift_left_1/2/3)

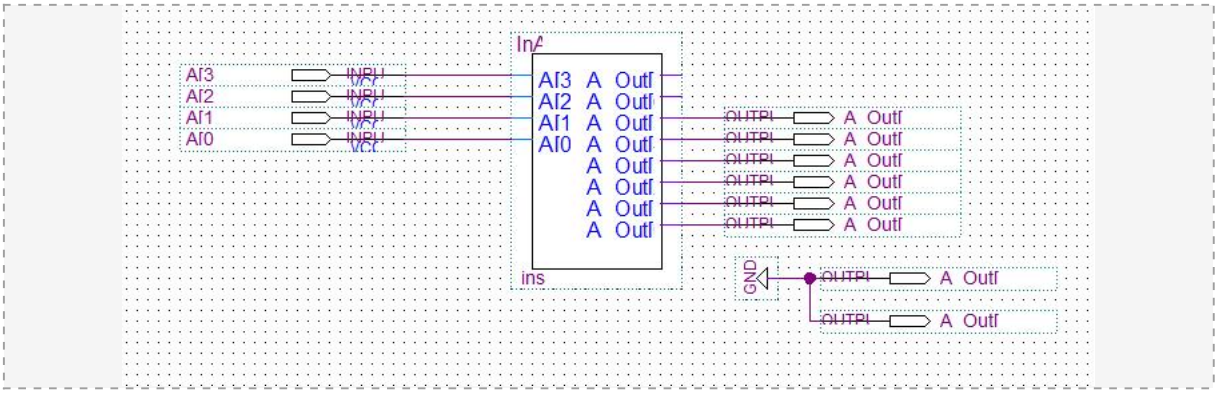
Mỗi bộ dùng InA để có A 8 bit rồi nối lệch dây sang trái, các bit thấp bị trống nối GND.

Bảng 4.2: Ngõ ra các bộ dịch

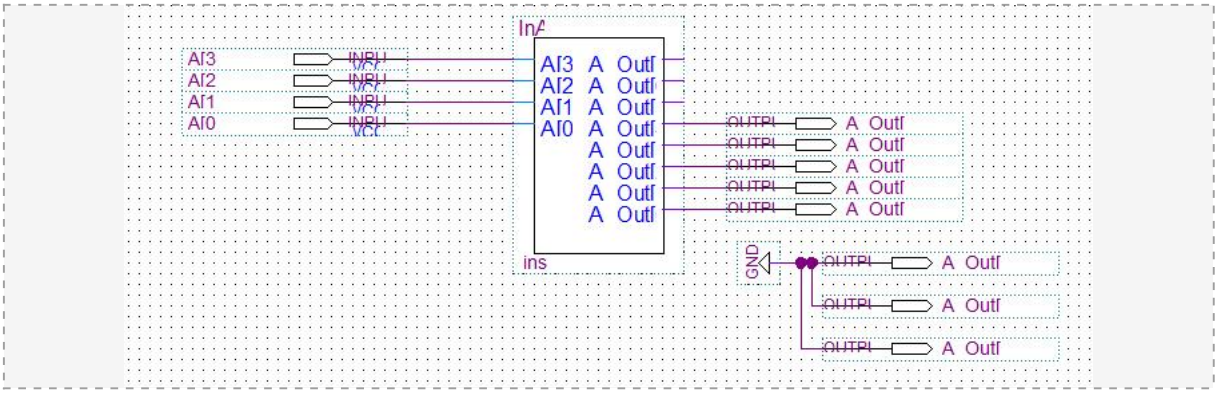
Khối	Phép toán	A_Out[7..0]
InA	A × 1	0 0 0 0 A3 A2 A1 A0
shift_left_1	A × 2	0 0 0 A3 A2 A1 A0 0
shift_left_2	A × 4	0 0 A3 A2 A1 A0 0 0
shift_left_3	A × 8	0 A3 A2 A1 A0 0 0 0



Hình 4.2: Bộ dịch trái 1 bit



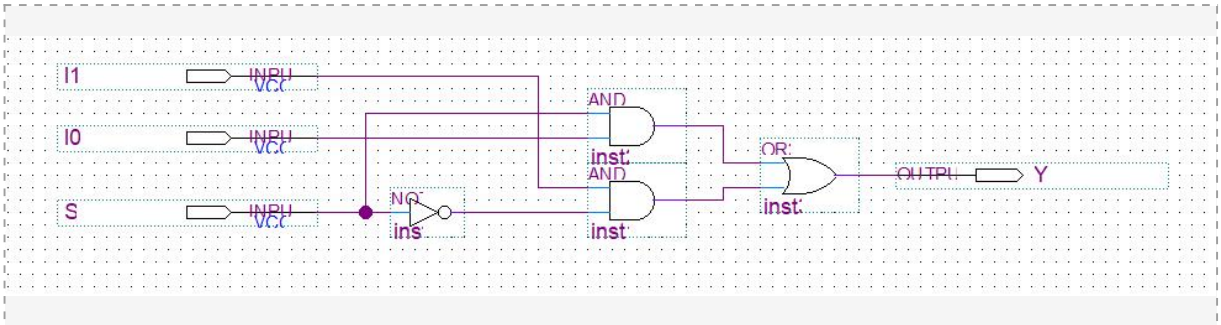
Hình 4.3: Bộ dịch trái 2 bit



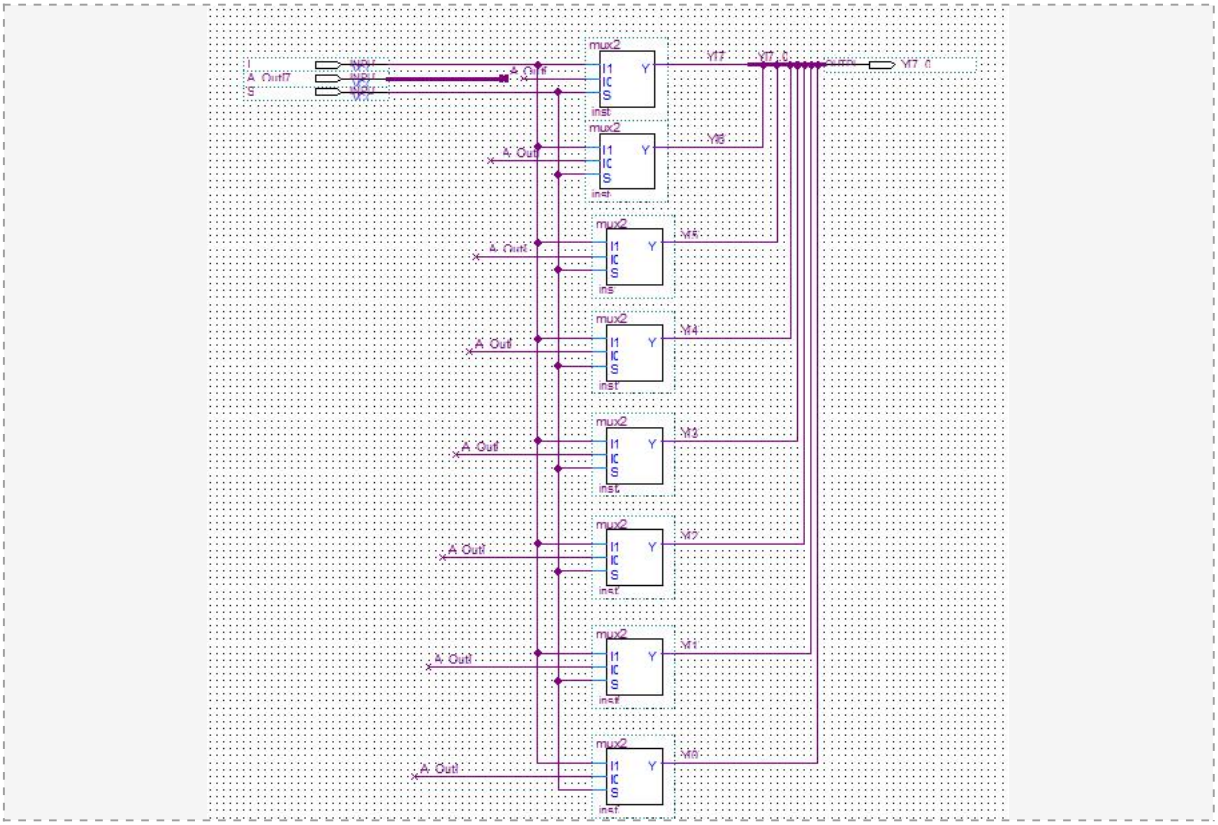
Hình 4.4: Bộ dịch trái 3 bit

4.4. Bộ chọn tích riêng phần (mux2_1_8bit)

Gồm 8 bộ mux2_1 dùng chung ngõ chọn S. Ngõ I nối GND, ngõ A_Out[7..0] nối số đã dịch: S = 0 → Y = 0000 0000, S = 1 → Y = A_Out. Bit B[i] nối vào S của bộ MUX tương ứng, nên ngõ ra chính là B[i]·(A << i).



Hình 4.5: Mạch MUX 2-1 của bộ nhân

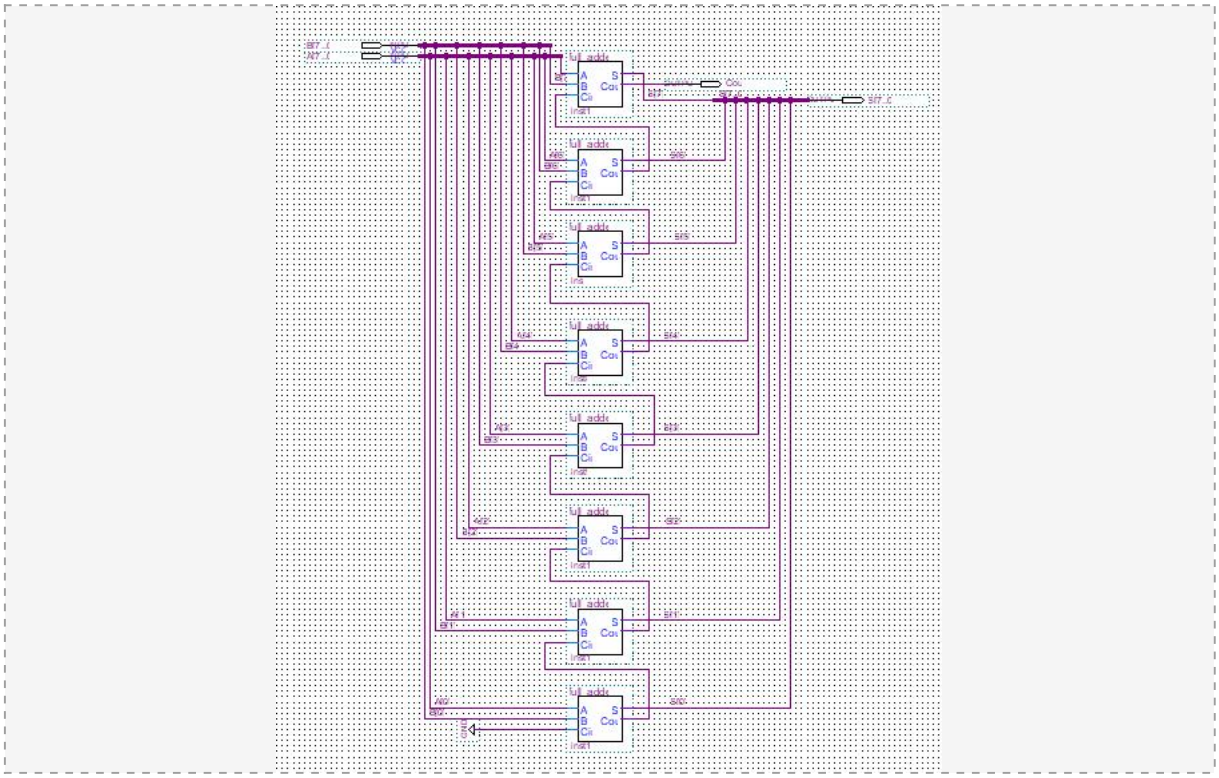


Hình 4.6: Bộ chọn tích riêng phần 8 bit

4.5. Bộ cộng 8 bit (full_adder_8bit)

Ghép nối tiếp 8 full_adder (ripple carry), C_{in} bit 0 nối GND. Bộ nhân dùng 3 bộ cộng này để cộng dồn 4 tích riêng phần.

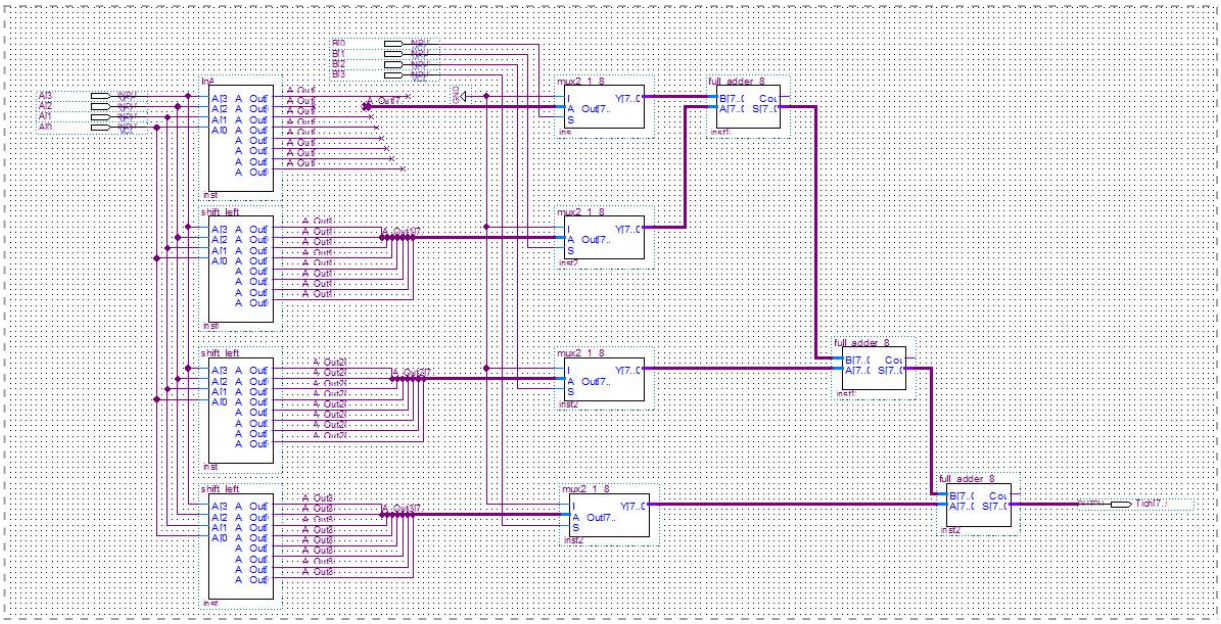
$$S[7..0] = A[7..0] + B[7..0]$$



Hình 4.7: Bộ cộng 8 bit

4.6. Mạch nhân hoàn chỉnh (nhan)

- 4 khối InA, shift_left_1, shift_left_2, shift_left_3 tạo A × 1, A × 2, A × 4, A × 8.
- 4 bộ mux2_1_8bit với S lần lượt là B[0], B[1], B[2], B[3] giữ lại hoặc xóa từng tích riêng phần P0 … P3.
- 3 bộ full_adder_8bit cộng dồn: (P0 + P1) → + P2 → + P3 = **Tich[7..0]**.



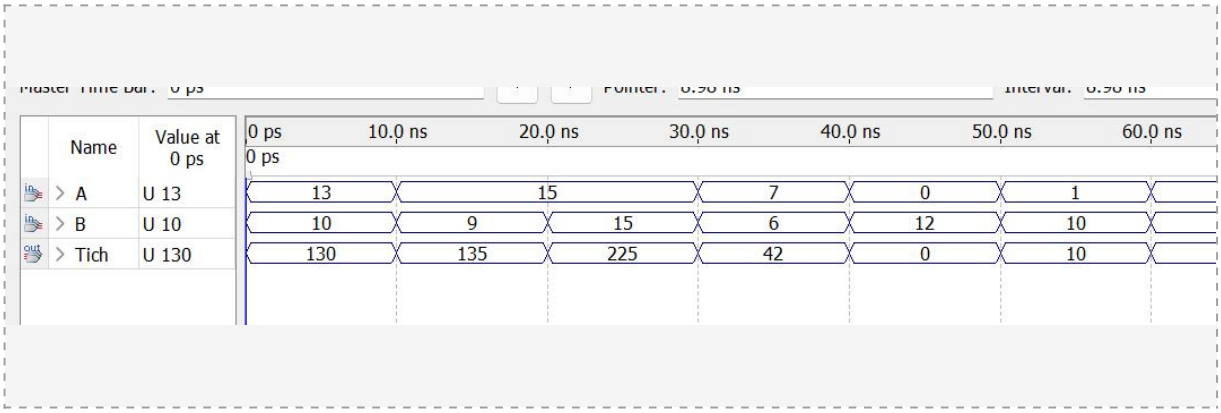
Hình 4.8: Mạch nhân hai số 4 bit hoàn chỉnh

4.7. Mô phỏng Functional

Mô phỏng trên mạch nhân (trước khi gắn khối hiển thị LED), kiểm tra cả trường hợp bằng 0 và trường hợp lớn nhất.

Bảng 4.3: Các trường hợp kiểm tra bộ nhân

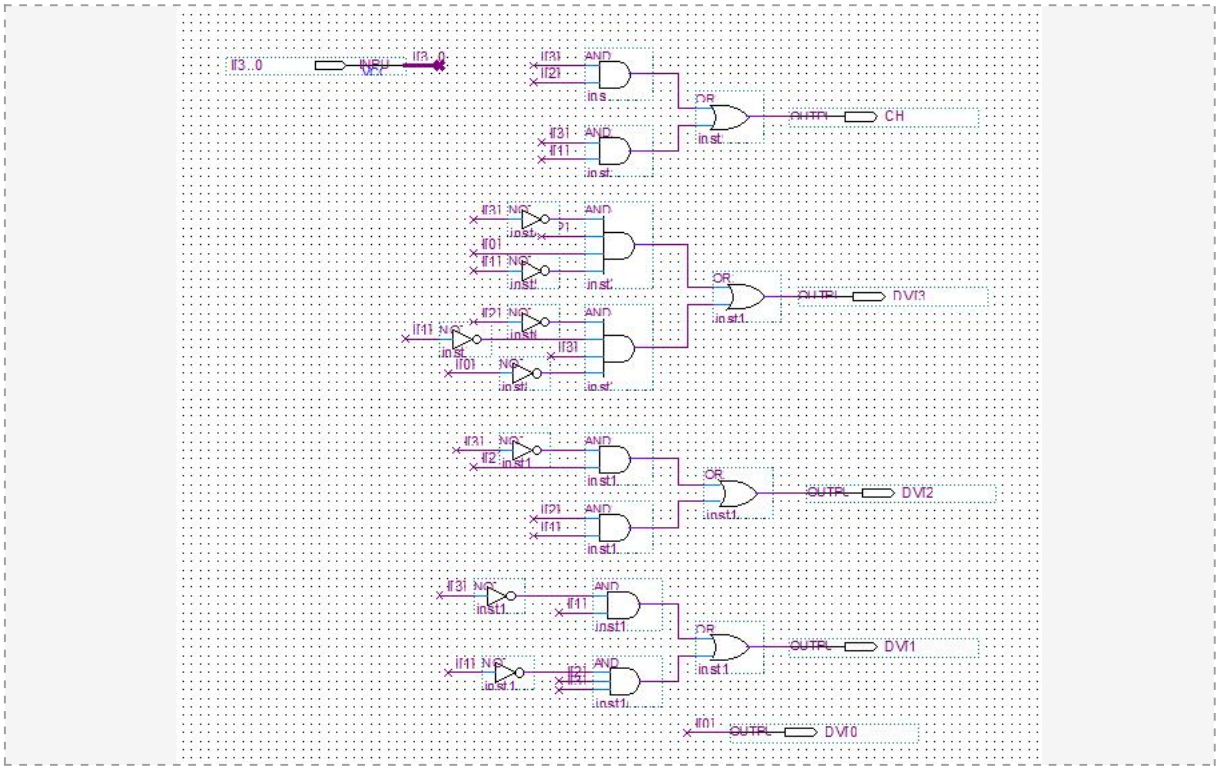
A	B	Tích mong đợi	Tích mô phỏng	Kết quả
1101 (13)	1010 (10)	1000 0010 (130)	1000 0010 (130)	Khớp
1111 (15)	1001 (9)	1000 0111 (135)	1000 0111 (135)	Khớp
1111 (15)	1111 (15)	1110 0001 (225)	1110 0001 (225)	Khớp
0111 (7)	0110 (6)	0010 1010 (42)	0010 1010 (42)	Khớp
0000 (0)	1100 (12)	0000 0000 (0)	0000 0000 (0)	Khớp



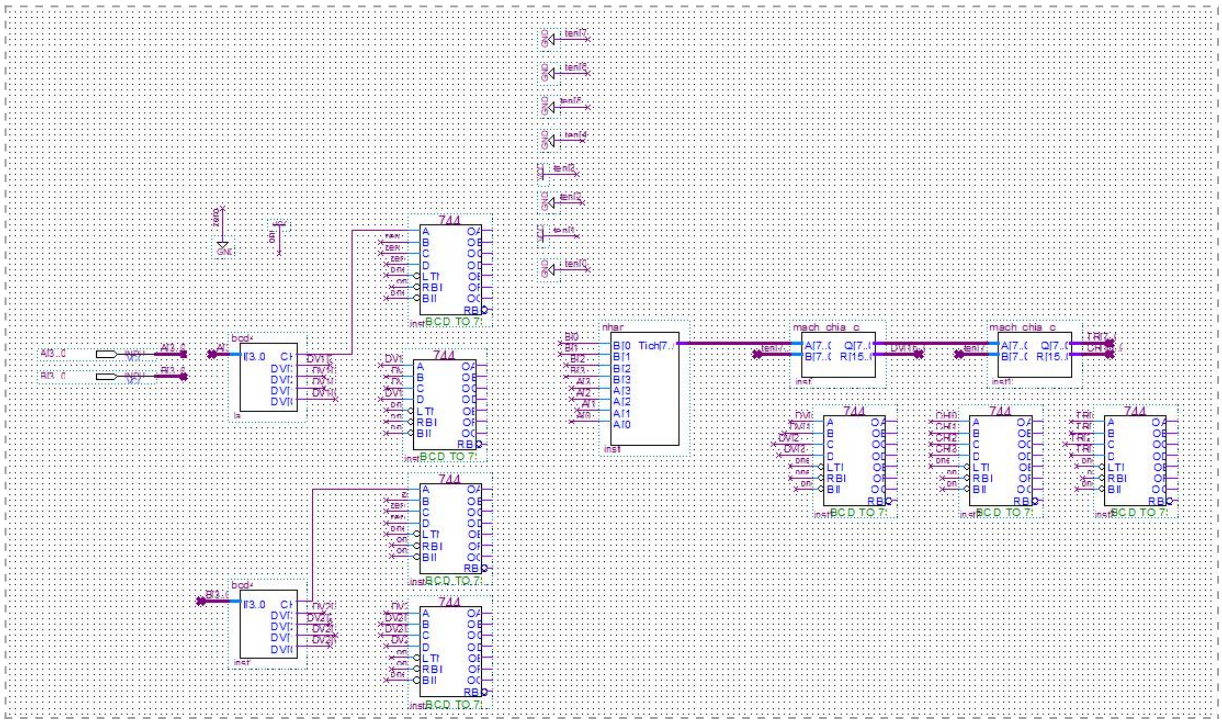
Hình 4.9: Waveform Functional – Bộ nhân 4 bit

4.8. Hiển thị LED và nạp kit DE2

Mạch hien_thi_led gắn bộ nhân với các khối **bcd4** (tách số thành hàng chục CH và hàng đơn vị DV) và bộ giải mã **7447** (BCD sang LED 7 đoạn), để A, B và tích hiển thị dạng thập phân trên LED 7 đoạn. Cách gán chân và các trường hợp thử được trình bày trong video.



Hình 4.10: Khối tách hàng chục – hàng đơn vị bcd4



Hình 4.11: Mạch hiển thị LED 7 đoạn

Link video: Video nạp bộ nhân trên kit DE2: https://drive.google.com/file/d/12bAdWvGBnQx7mrrkStlXNDGZXHwpbM6j/view?usp=sharing

5. KẾT LUẬN

- Đã thiết kế hoàn chỉnh **ALU 16 bit** với 8 phép toán đúng thứ tự Opcode 0 → 7, gồm khối AU (cộng, cộng 1, trừ, trừ 1), khối LU (AND, OR, NAND, XOR) và MUX chọn theo Opcode[2], vẽ hoàn toàn bằng cổng logic.
- Đã thiết kế **bộ nhân hai số 4 bit bất kỳ** (mở rộng so với yêu cầu nhân với số cuối MSSV) theo hướng bộ dịch + MUX + bộ cộng, kết quả 8 bit.
- Mô phỏng Functional cho kết quả đúng ở tất cả trường hợp kiểm tra.
- Cả hai mạch đã nạp và chạy đúng trên kit DE2; riêng ALU giữ cố định 10 bit cao và chỉ thay đổi 6 bit thấp do kit không đủ công tắc.
- **Hạn chế và hướng cải tiến:** bộ cộng/trừ 16 bit dùng hai chuỗi riêng cho A ± B và A ± 1 nên tốn phần cứng; có thể dùng một chuỗi duy nhất với MUX chọn B hoặc 1 ở ngõ vào, hoặc gộp cộng và trừ bằng phép bù 2 (B XOR S, C_{in} = S). Bộ cộng ripple carry có độ trễ tăng theo số bit, có thể thay bằng bộ cộng nhìn trước số nhớ (CLA).